



← Other posts



Can Machine Learning Predict Bitcoin? Seven Models, One Honest Answer

by [Jerome Etienne](#) on Mon May 25 2026

bitcoin_ml

Bitcoin price prediction is the world's most popular machine learning failure. Every quant retail trader, every grad student, every YouTube channel has tried it. The graveyard is enormous. So why bother?

Can Machine Learning Predict Bitcoin?

Seven models, one data slice, one honest scoreboard.
From naive last-value to fine-tuned foundation models.

jerome etienne • github.com/jeromeetienne/transformer_bitcoin_ai

The full source code for this series is available at
github.com/jeromeetienne/transformer_bitcoin_ai.

Because if you build the experiment honestly - same data slice, same metrics, same evaluation rules - you learn something that the YouTube videos never tell you. Not "can a model predict Bitcoin", which is the wrong question. The right question is: **across the full spectrum of model classes, which ones actually clear the random-walk floor, and by how much?**

This series runs that experiment. Seven models, from a zero-parameter naive predictor to a 200-million-parameter foundation model. Same Bitcoin, same window, same scoreboard. The result is the most useful kind of finding: not "Bitcoin is unpredictable" (it isn't), and not "deep learning wins" (it doesn't), but a

clear gradient across the ladder of model classes -
and a handful of lessons that generalize to any
forecasting problem.

What the project is

Seven experiments live under [experiments/](#) in the repository. Each one is a folder with a config, a `run.py`, a results directory, and a report. They share a metric module, a data loader, and an evaluation harness, so the numbers are directly comparable.

The ladder, in order:

- **01_baseline** ([01_baseline](#)) - naive last-value.
Predicts that the next price equals the current price. Zero parameters. This is the floor every later model must clear.
- **02_arima** ([02_arima](#)) - classical statistical model on differenced prices. Three to seven parameters depending on order.
- **03_xgboost** ([03_xgboost](#)) - gradient boosting on 31 engineered features (lagged log-returns, rolling moments, OHLC summaries).

- **04_lstm** ([04_lstm](#)) - recurrent neural network on raw sequences. Two stacked LSTM layers, thousands of weights.
- **05_transformer** ([05_transformer](#)) - Temporal Fusion Transformer. Attention plus variable selection plus gated residuals. Tens of thousands of weights.
- **06_pretrained** ([06_pretrained](#)) - zero-shot foundation models. Chronos-2 (28M and 120M parameters) and TimesFM 2.5 (200M parameters). Models that have never seen Bitcoin.
- **07_finetuned** ([07_finetuned](#)) - the same foundation models, but with `fit()` actually updating weights on a long Bitcoin train slice.

Each experiment gets one article in this series. This article is the curtain-raiser: the question, the data, the metrics, and a preview of where things go.

The data

Bitcoin against USDT on Binance, sampled every four hours. One row per bar, with OHLCV columns.

- **Symbol:** BTCUSDT
- **Interval:** 4 hours
- **Train window:** varies by experiment. For 01 through 05 it is roughly 2024-01-01 to 2024-08-01. For 06 *pretrained the train window extends back to 2023-01-01*. For 07 *finetuned the canonical variant trains on 2020-01-01 to 2024-08-01 (about 4.7 years, ~10 000 bars)*.
- **Test window:** 2024-10-01 to 2024-12-01 UTC. The same window across every experiment. 366 bars at 4h. This is the post-election Bitcoin rally - a strongly trending regime.

Same test window across every experiment is the load-bearing design choice. It means every leaderboard row in this series is apples-to-apples on the metric side. The train slice differs (06 and 07 use longer histories), but the test bars are the same.

The target and the prediction

The target is the next bar's log-return:

$$r_T = \log(\text{close}_T / \text{close}_{\{T-1\}})$$

Log-returns are stationary, roughly symmetric, and scale-free. They are the standard target in financial time-series forecasting for good reason: predicting the raw price level produces models that "learn" the long-run upward trend and look great in-sample, then fall apart out-of-sample. Differencing strips that out.

Every model predicts r_T (or in the case of the naive baseline, predicts no change). The price prediction is reconstructed:

$$\text{close}_{\text{pred}} = \text{close}_{\{T-1\}} * \exp(r_{\text{pred}})$$

That way the leaderboard's MAE / RMSE / MAPE are all in dollars, and directly comparable to the raw close prices a reader can sanity-check.

The metrics

Six numbers populate every results file. Each measures something different.

- **MAE (Mean Absolute Error)** - average dollars wrong per bar. The closest-to-the-price metric. Reported in USD.
- **RMSE (Root Mean Squared Error)** - the square root of the average squared error. Penalizes big misses much more than small ones. Also in USD.
- **MAPE (Mean Absolute Percentage Error)** - the same as MAE but as a percentage of the actual price. Comparable across regimes where the absolute price level changes.
- **Directional accuracy** - the fraction of test bars where `sign(pred - ref) == sign(actual - ref)`. Half a coin flip is 0.5; anything above means the model has some directional signal.
- **Cumulative return** - the gross return of a long/flat strategy that goes long when `pred > ref` and stays flat otherwise. No shorting.
- **Annualized Sharpe** - the per-bar Sharpe ratio of that strategy, annualized by multiplying by `sqrt(periods_per_year)`. For 4h bars that's `sqrt(2190)`.

Every metric is computed by

`metrics.json` is produced by the same code. No model gets to grade its own homework.

The evaluation philosophy

A few rules that hold across every experiment:

1. **Single train/test split.** No rolling-origin cross-validation. The test window is fixed at 2024-10-01 to 2024-12-01. Every model is evaluated on the same 366 bars.
2. **Walk-forward without re-fitting.** For each test bar, the model gets the entire prior history up to that bar and produces a one-step-ahead forecast. But the model's parameters are frozen at train-time values. No online learning, no re-estimation between bars.
3. **Long/flat strategy.** When the model predicts up ($\text{pred} > \text{ref}$), the strategy is long for that bar. When it predicts down or no-change, the strategy is flat. No shorting, no leverage, no transaction costs, no slippage.

4. **Verbatim numbers.** Every value in every report is copied directly from the experiment's `metrics.json` or `sweep.csv`. Nothing is recomputed, rounded, or interpolated for storytelling.

These choices keep the comparison clean. They are also why each result includes a "regime caveat": the test window is two months of a strong uptrend, and every positive Sharpe in the series is conditional on that regime.

The recurring lessons (without spoiling the punchlines)

Six observations show up across the seven experiments. Each gets explored in detail in the article it surfaces from, but they are worth previewing:

1. **Capacity does not equal skill.** Bigger models do not always score better. On limited data, they often score worse.

2. **Point error and trading metrics measure**

different things. A model can have a worse MAE than the naive baseline and still beat it on Sharpe. (XGBoost does this.)

3. **In-sample selection does not rank out-of-**

sample Sharpe. AIC, BIC, validation loss - none of them reliably pick the configuration that wins on the trading metric. (Both ARIMA's order sweep and the foundation-model fine-tuning sweep make this visible.)

4. **NaN is a feature.** When the metric is NaN, the model is telling you it has no opinion. The naive baseline gets a NaN directional accuracy by design - it never predicts a direction.

5. **Zero-shot transfer is harder than the**

marketing. A 200-million-parameter foundation model trained on hundreds of millions of time series loses to a 3-parameter ARIMA on Bitcoin if it never sees any.

6. **Every positive Sharpe is regime-conditional.**

The test window is the post-election 2024 rally.

None of the models in this series have been tested out-of-regime. The numbers should be read as

"skill conditional on this regime", not "skill in general".

How to read this series

The seven model articles can be read in order (they build a ladder) or out of order (each one is self-contained: what the model does, how it was set up, what it found). The narrative arc - and the lesson - is in the order.

A reader who wants the punchline can jump to the [cross-experiment synthesis](#) in the reports folder. A reader who wants to understand why the punchline is what it is should read the articles.

Either way, every model in this series was built with the same scoreboard in mind. The goal was never to win at predicting Bitcoin. The goal was to learn what each model class actually does to a hard problem - and to read the results honestly, including the failures.

Every experiment is one make target:

```
make 01_baseline      # naive last-value
make 02_arima         # ARIMA(p, d, q)
make 03_xgboost       # gradient boosting
make 04_lstm          # recurrent net
make 05_transformer   # TFT
make 06_pretrained    # zero-shot Chronos
make 07_finetuned     # fine-tuned Chronos
```

Each target writes its `metrics.json` to `experiments/0N_*/results/btc_4h_2024/`. The reports under [docs_ml/reports/](#) read those files and summarize. The articles in this series read the reports.

If a number in any of these articles does not match the on-disk `metrics.json` for the corresponding experiment, the on-disk value wins. The articles are interpretations; the artifacts are authoritative.

The seven model articles follow.

by [Jerome Etienne](#) on Mon May 25 2026